Reconocimiento de Patrones — Examen Intermedio (Examen B)

Universidad de Sonora · Licenciatura en Ciencias de la Computación

Alumno: Owen Adiel Solis Zatarain
Expediente: 223201053

Semestre 2026-2

Parámetros personales del examen:	
Expediente completo (semilla):	`223201053`
Último dígito (a):	$a = 3$
Penúltimo dígito (b):	$b = 5$

1. Problema 1: El reporte demasiado bueno

1.1. Parte teórica

(a) Copias en la prueba (6 pts)

1. **Fracción de renglones positivos después de repetir:**

   En el conjunto de datos original existen $N_{\text{neg}} = 39{,}922$ llamadas sin contratación (clase negativa) y $N_{\text{pos}} = 5{,}289$ llamadas con contratación (clase positiva). En el código de la consultora se ejecuta `pd.concat([X] + [X[pos]] * 6)`, lo que concatena la tabla original (que ya contiene una vez a los positivos) con 6 copias adicionales de los casos positivos. En consecuencia, cada renglón positivo aparece 7 veces en total:

   $$N_{\text{pos, nuevo}} = 7 \times 5{,}289 = 37{,}023, \qquad N_{\text{total}} = 39{,}922 + 37{,}023 = 76{,}945$$

   La nueva fracción de renglones positivos en la tabla modificada es:

   $$f_+ = \frac{N_{\text{pos, nuevo}}}{N_{\text{total}}} = \frac{37{,}023}{76{,}945} \approx 0.48116 \quad (48.12\,\%)$$

   Esto distorsiona la distribución real del problema (donde originalmente solo el 11.70 % contrató), haciendo que casi la mitad del conjunto de datos parezca corresponder a contrataciones exitosas.

2. **Probabilidad de que un positivo en prueba tenga al menos una copia en entrenamiento:**

   Si tomamos un renglón positivo que cayó en el conjunto de prueba, sabemos que en el conjunto de datos inflado existen otras 6 copias idénticas de ese mismo cliente. Bajo el supuesto de que cada copia se asigna de forma independiente a entrenamiento con probabilidad 0.8 (80 %) o a

prueba con probabilidad 0.2 (20 %), el único escenario en el que dicho renglón de prueba *no* tiene ninguna copia idéntica en el entrenamiento es que las otras 6 copias hayan caído todas también en el conjunto de prueba:

$$P(\text{ninguna copia en ent.}) = (0.2)^6 = 0.000064$$

Por complemento, la probabilidad de que tenga al menos una copia idéntica en el conjunto de entrenamiento es:

$$q = P(\text{al menos una copia en ent.}) = 1 - (0.2)^6 = 0.999936 \quad (99.9936\,\%)$$

Es decir, prácticamente la totalidad de los casos positivos evaluados en la prueba ya fueron vistos de manera idéntica durante el entrenamiento.

3. **Exactitud medida por la consultora como función de p:**

   Podemos particionar el conjunto de prueba en dos grupos disjuntos:

   - Los renglones positivos que tienen al menos una copia idéntica en el entrenamiento, los cuales representan una fracción $c = f_+ \cdot q$ del conjunto de prueba. Como el modelo memoriza estos casos vistos en entrenamiento, acierta siempre en ellos (exactitud igual a 1).
   - El resto de los renglones del conjunto de prueba (todos los negativos más la fracción minúscula de positivos sin copia en entrenamiento), que representan una proporción $1-c$, donde el modelo tiene su exactitud real p.

   Calculando la proporción c:

$$c = f_+ \cdot q = \frac{37{,}023}{76{,}945} \times 0.999936 \approx 0.48116 \times 0.999936 \approx 0.4811$$

   Por la ley de probabilidad total, la exactitud que mide la consultora en función de p es:

$$\text{Exactitud}(p) = c \cdot (1) + (1 - c) \cdot p \approx 0.4811 + 0.5189\,p$$

   Este resultado demuestra que el código de la consultora tiene garantizado un piso de 48.11 % de exactitud únicamente por memorizar copias repetidas: incluso si el modelo tuviera una exactitud real mediocre o equivalente a lanzar una moneda ($p = 0.5$), el reporte mediría una exactitud inflada de $0.4811 + 0.5189(0.5) \approx 74.06\,\%$.

(b) Elegir con la prueba (5 pts)

Sean $\hat{p}_1, \ldots, \hat{p}_m$ las exactitudes medidas en los n datos de prueba de los m modelos independientes, todos con la misma exactitud real p. Por la propiedad **(P2)**, cada $\hat{p}_j$ sigue aproximadamente una distribución normal con media p y desviación estándar $\sigma = \sqrt{p(1-p)/n}$, por lo que podemos escribirla en términos de una variable aleatoria normal estándar $Z_j \sim \mathcal{N}(0,1)$ independiente como:

$$\hat{p}_j = p + \sqrt{\frac{p(1-p)}{n}}\, Z_j$$

Al elegir el modelo con mayor exactitud en el conjunto de prueba, dado que p y $\sqrt{p(1-p)/n}$ son constantes positivas que no dependen de j, se tiene que:

$$\max_{1 \le j \le m} \hat{p}_j = p + \sqrt{\frac{p(1-p)}{n}} \left(\max_{1 \le j \le m} Z_j \right)$$

Tomando el valor esperado en ambos lados por linealidad de la esperanza y aplicando la propiedad **(P1)** ($\mathbb{E}[\max_j Z_j] \approx \sqrt{2\ln m}$):

$$\mathbb{E}\left[\max_{1\le j\le m} \hat{p}_j\right] = p + \sqrt{\frac{p(1-p)}{n}}\,\mathbb{E}\left[\max_{1\le j\le m} Z_j\right] \approx p + \sqrt{\frac{p(1-p)}{n}}\sqrt{2\ln m}$$

Por lo tanto, el sesgo o sobreestimación esperada respecto a la exactitud real p es:

$$\text{Sesgo} = \mathbb{E}\left[\max_{1\le j\le m} \hat{p}_j\right] - p \approx \sqrt{\frac{p(1-p)}{n}}\sqrt{2\ln m}$$

Sustituyendo los valores numéricos con $p = 0.8$ (de modo que $p(1-p) = 0.8 \times 0.2 = 0.16$):

- **Caso de la consultora ($m = 4$ profundidades probadas, $n = 0.2 \times 76{,}945 = 15{,}389$ datos de prueba):**

$$\text{Sesgo}_{\text{consultora}} \approx \sqrt{\frac{0.8(0.2)}{15{,}389}}\sqrt{2\ln(4)} \approx 0.003224 \times 1.665109 \approx 0.0054 \quad (0.54\,\%)$$

- **Caso personal ($a = 3, b = 5 \implies m = 10^{1+(3 \bmod 3)} = 10^1 = 10$ modelos, $n = 100(5+1) = 600$ datos de prueba):**

$$\text{Sesgo}_{\text{personal}} \approx \sqrt{\frac{0.8(0.2)}{600}}\sqrt{2\ln(10)} \approx 0.016330 \times 2.145966 \approx 0.0350 \quad (3.50\,\%)$$

¿Cuándo importa este problema y cuándo no? A partir de la expresión $\sqrt{p(1-p)/n}\sqrt{2\ln m}$, observamos que la sobreestimación disminuye con la raíz del tamaño de la muestra de prueba ($1/\sqrt{n}$) y crece lentamente con el número de modelos evaluados ($\sqrt{2\ln m}$). Por ello, este problema **importa mucho** cuando el conjunto de prueba n es pequeño (como en el caso personal con $n = 600$, donde infla la exactitud en 3.50 puntos porcentuales) y cuando se prueban decenas o cientos de combinaciones de hiperparámetros m. En cambio, **casi no tiene impacto numérico** cuando n es muy grande y se comparan pocos modelos (como en el código de la consultora con $n = 15{,}389$ y $m = 4$, donde apenas aporta 0.54 %), aunque metodológicamente sigue siendo una mala práctica.

(c) Seleccionar atributos con ruido (6 pts)

Supongamos que se tienen $n = 50$ datos y $m = 5{,}000$ atributos que son puro ruido, independientes de la clase. Por la propiedad **(P3)**, la correlación muestral r_j entre cada atributo j y la clase sigue aproximadamente una distribución normal con media 0 y desviación estándar $1/\sqrt{n}$, por lo que podemos expresarla como $r_j = \frac{1}{\sqrt{n}} Z_j$, con $Z_j \sim \mathcal{N}(0,1)$. Aplicando la propiedad **(P1)** para estimar el máximo entre los $m = 5{,}000$ atributos:

$$\mathbb{E}\left[\max_{1\le j\le 5000} r_j\right] \approx \frac{1}{\sqrt{50}}\sqrt{2\ln(5{,}000)} \approx 0.141421 \times 4.127273 \approx 0.5837$$

(Si consideramos ambos extremos de la distribución por el valor absoluto $|r_j|$, equivalente a $2m = 10{,}000$ colas, la correlación máxima esperada sube a $\frac{1}{\sqrt{50}}\sqrt{2\ln(10{,}000)} \approx 0.6070$). Es decir, aunque todos los atributos sean ruido puro, por simple azar muestral al tener pocos renglones ($n = 50$) y miles de columnas (5,000), encontraremos atributos con correlaciones aparentes cercanas a 0.60.

¿Por qué infla la validación cruzada elegir antes? Si seleccionamos los 20 atributos más correlacionados utilizando *todos* los 50 datos antes de partir, estamos filtrando aquellas columnas de ruido que, por casualidad, coincidieron con las etiquetas y de esos 50 renglones específicos. Cuando posteriormente dividimos esos mismos 50 renglones en pliegues de entrenamiento y validación cruzada, los datos del pliegue de validación ya intervinieron en la elección de los 20 atributos. El clasificador aprende esa correlación espuria que ya está presente en toda la muestra y obtiene una exactitud muy superior al 50 %, a pesar de que ante datos nuevos su desempeño real sería de 50 %.

Forma correcta de hacerlo: La selección de atributos debe realizarse *dentro* de cada pliegue de la validación cruzada, ajustando el selector (`SelectKBest`) únicamente con los datos de entrenamiento de ese pliegue y aplicándolo después al pliegue de validación (en `scikit-learn`, encapsulando el selector y el clasificador dentro de un `Pipeline`).

(d) ¿Es trampa la duración? (4 pts)

Definición de fuga de información temporal: Sea t_0 el instante en el que se ejecuta el modelo en producción para tomar una decisión sobre una observación. Decimos que una variable X_k presenta *fuga de información* respecto a t_0 si su valor solo se genera, determina u observa en un instante $t > t_0$ (posterior a la toma de decisión o como consecuencia del evento objetivo Y).

¿Es `duration` una fuga de información? Depende de la etapa operativa en la que se utilice el modelo:

- **Uso pre-llamada (t_0 = antes de marcar al cliente): Sí es fuga.** Si el objetivo del banco es decidir a qué clientes de su cartera vale la pena llamar en la campaña, en el instante t_0 la llamada aún no ha ocurrido y es imposible conocer cuántos segundos durará. Además, en la práctica de un centro de llamadas, aunque una llamada larga no garantiza por sí sola que el cliente contrate (una persona puede hacer preguntas durante mucho tiempo y no comprar), una llamada muy corta (por ejemplo, menor al tiempo mínimo indispensable para explicar el producto y tomar los datos del contrato, que suele ser de al menos un minuto) descarta casi con certeza la contratación. Incluir `duration` le permite al modelo descartar trivialmente todas las llamadas cortas usando información del futuro.

- **Uso post-llamada (t_0 = después de finalizar la llamada): No es fuga.** Si el modelo se emplea en una segunda etapa operativa después de haber realizado el contacto telefónico, `duration` ya es un dato conocido ($t \leq t_0$). Por ejemplo, se puede ordenar el histórico de llamadas exitosas para establecer el tiempo mínimo requerido para concretar una contratación: si el cliente no contesta (0 s), se mantiene como potencial para reintentar después; si contesta pero cuelga por debajo de ese umbral mínimo, se descarta; y si supera el umbral sin haber cerrado la contratación en ese instante, se prioriza para seguimiento o auditoría de calidad.

¿Qué otras variables tienen el mismo problema? En el escenario pre-llamada (antes de iniciar los contactos de la campaña para un cliente), presentan el mismo problema las variables registradas al cierre del contacto de la campaña actual:

- `campaign`: representa el número total de llamadas realizadas al cliente durante esta campaña (incluyendo el último contacto). Antes de empezar a llamarle, es imposible saber cuántos intentos tomará localizarlo o convencerlo.

- `day` y `month`: registran el día y el mes en que ocurrió el *último* contacto de la campaña, fecha que tampoco se conoce antes de efectuar las llamadas.

(e) El tiempo (4 pts)

Por qué falla una partición aleatoria: Las 45,211 llamadas se realizaron entre mayo de 2008 y noviembre de 2010, un periodo marcado por fuertes cambios macroeconómicos (la crisis financiera de 2008 frente a la estabilización posterior). Una partición aleatoria mezcla llamadas de todas las fechas tanto en entrenamiento como en prueba, lo cual distorsiona la estimación en dos sentidos:

- **Puede ser demasiado optimista:** porque para predecir el comportamiento de clientes en un mes determinado (por ejemplo, mediados de 2010), el modelo ya fue entrenado con llamadas ocurridas en ese mismo mes y bajo las mismas condiciones económicas, algo imposible en la vida real donde el futuro aún no ocurre.
- **Puede ser pesimista o contaminar el modelo:** porque los meses más críticos de 2008 (cuando la economía estaba en crisis y el desempeño de las campañas era anormalmente bajo) se mezclan con los datos recientes, contaminando tanto el aprendizaje de las tendencias actuales como la evaluación de lo que ocurrirá en la siguiente campaña.

Partición propuesta que imita el uso real: Para evaluar honestamente cómo funcionará el modelo en la próxima campaña, debemos respetar el orden cronológico:

1. **Corte cronológico para prueba:** Reservar el último bloque temporal de los datos (por ejemplo, el último 20 % de los renglones, correspondiente a los meses más recientes de 2010) como conjunto de prueba final.
2. **Validación con ventana deslizante (*sliding window*):** En lugar de acumular todos los datos desde mayo de 2008, entrenar y validar avanzando por bloques temporales mediante una ventana móvil de tamaño fijo que utilice únicamente los meses recientes previos al bloque que se desea predecir, descartando así los meses antiguos afectados por eventos económicos atípicos como la crisis de 2008.

1.2. Carta al banco (Inciso f de la parte práctica)

Estimado Banco:

El modelo que les entregó la consultora exageraba lo que pueden esperar de su próxima campaña telefónica. Su 93 % de aciertos no significa mucho: como solo el 12 % de los clientes contrató, un modelo que dijera "no" a todos ya acertaría el 88 %. Revisamos el código y encontramos tres errores principales:

- **Usaba la duración de la llamada.** El modelo usaba cuánto duró cada llamada, pero ese dato solo se conoce *después* de llamar. Es como adivinar quién va a contratar sabiendo ya si la conversación fue larga. Cuando quitamos ese dato, el desempeño cayó notablemente. Lo mismo pasa con el número de llamadas de la campaña y la fecha del último contacto.
- **Mezcló las fechas.** Las llamadas van de 2008 a 2010, y la consultora las revolvió antes de probar el modelo. Así, el modelo aprendía con llamadas de 2010 para "predecir" llamadas de 2008, algo que en la vida real nunca va a pasar: ustedes siempre van a predecir la próxima campaña con información del pasado. Además, el porcentaje de clientes que contrata cambió mucho con el tiempo (en los primeros meses contrataba cerca del 3 % y al final casi la mitad), así que mezclar fechas esconde ese cambio. Este fue el error que más infló el resultado.
- **Calificó al modelo con datos que ya conocía.** Para compensar que pocos clientes contratan, duplicaron seis veces las llamadas con contratación, y lo hicieron antes de separar los datos de entrenamiento y de prueba. También eligieron las variables y el ajuste final mirando los datos de prueba. Es como presentar un examen después de haber visto las respuestas.

¿Qué desempeño es realista? Corregimos todo y probamos el modelo como se va a usar: entrenándolo con las llamadas más antiguas y midiéndolo con las más recientes, que nunca vio. El modelo no sirve para decir con certeza quién va a contratar, pero sí para **ordenar** a sus clientes de mayor a menor probabilidad de contratar. Si llaman primero al 10 % de clientes que el modelo pone hasta arriba, cerca del **70 %** contrata (entre 67 % y 73 %), contra el **32 %** si llaman al azar en ese mismo periodo. Es decir, consiguen alrededor de **2.2 veces más contrataciones por llamada**.

Lo que recomiendo:

1. Usar el modelo para decidir *en qué orden* llamar a sus clientes, empezando por los de mayor probabilidad, y no como una respuesta de "sí" o "no".
2. Actualizar el modelo antes de cada campaña usando solo las llamadas más recientes. Encontramos que entrenar con las llamadas recientes funciona mejor que usar todo el historial, porque el comportamiento de los clientes cambia con el tiempo.
3. Pedirle a cualquier proveedor que demuestre su modelo con campañas *posteriores* a las que usó para construirlo, y desconfiar de resultados casi perfectos.
4. Medir los resultados reales de cada campaña y compararlos con lo esperado, para detectar a tiempo si el modelo deja de funcionar.

Atentamente,
Owen Adiel Solis Zatarain

1.3. Bitácora de decisiones — Problema 1